CAPSTONE PROJECT

Agentic AI

Studi Kasus:

KnowledgeHub Assistant

Membangun Multi-Agent RAG Chatbot untuk
Sistem Tanya-Jawab Cerdas Berbasis Dokumen

Program	Agentic AI
Tipe Tugas	Final Project — Individu
Tools Utama	LangChain, LangGraph, OpenAI, Qdrant Cloud, Python (Terminal)
Durasi Pengerjaan	10 Hari Kerja

1. Overview & Latar Belakang

1.1 Konteks Program

Setelah menyelesaikan pembelajaran di program Agentic AI, siswa telah melalui perjalanan belajar yang komprehensif — mulai dari dasar-dasar text representation, pengenalan Large Language Model (LLM), arsitektur Retrieval-Augmented Generation (RAG), hingga penerapan agentic workflow untuk kasus nyata seperti Text-to-SQL.

Final Project ini merupakan puncak dari perjalanan tersebut. Siswa ditantang untuk tidak sekadar memahami konsep secara teoritis, tetapi mampu mengintegrasikan seluruh komponen yang telah dipelajari ke dalam satu aplikasi yang fungsional, terstruktur, dan dapat digunakan secara nyata.

1.2 Profil Studi Kasus

KnowledgeHub adalah sebuah platform internal milik perusahaan teknologi skala menengah yang menyimpan ribuan dokumen: mulai dari dokumentasi produk, SOP operasional, kebijakan perusahaan, hingga laporan teknis. Selama ini, karyawan kesulitan menemukan informasi yang relevan karena dokumen tersebar di berbagai folder tanpa sistem pencarian yang cerdas.

Kebutuhan yang muncul adalah sebuah chatbot yang tidak hanya bisa menjawab pertanyaan, tetapi juga bisa memahami konteks percakapan, mengambil informasi dari sumber yang tepat, dan mengarahkan pertanyaan ke agen yang paling kompeten — persis seperti seorang asisten yang terlatih dan berpengetahuan luas.

1.3 Permasalahan yang Diselesaikan

Departemen	Masalah Utama	Dampak
Karyawan Umum	Tidak dapat menemukan dokumen yang relevan secara cepat	Pekerjaan terhambat, waktu terbuang
Tim HR & Legal	Pertanyaan berulang seputar kebijakan dijawab manual oleh tim	Beban kerja meningkat, jawaban tidak konsisten
Tim Teknis	Dokumentasi teknis sulit diakses saat debugging atau onboarding	Onboarding lambat, potensi error meningkat
Manajemen	Tidak ada laporan insight tentang pertanyaan yang paling sering diajukan	Keputusan berbasis data sulit dilakukan

1.4 Misi Final Project

Misi Final Project

Kamu adalah seorang AI Engineer yang baru bergabung di tim KnowledgeHub.
Tugasmu adalah merancang dan membangun sebuah multi-agent RAG chatbot menggunakan LangChain dan LangGraph yang mampu menjawab pertanyaan berdasarkan dokumen internal perusahaan — secara akurat, kontekstual, dan dengan transparansi proses berpikir yang jelas.

Seluruh sistem yang dibangun harus dapat dijalankan melalui antarmuka Streamlit yang clean dan interaktif, sehingga pengguna non-teknis pun dapat menggunakannya dengan mudah.

2. Lingkup & Ketentuan Pengerjaan

2.1 Ketentuan Umum

- Project dikerjakan secara individu.
- Data dokumen yang digunakan boleh berupa dummy data selama realistis dan representatif — minimal 3 dokumen dengan total isi yang bermakna.
- Tidak semua komponen harus sempurna — kedalaman implementasi dan pemahaman konsep lebih dihargai daripada sekadar kuantitas fitur.
- Setiap komponen wajib dapat didemonstrasikan secara end-to-end dengan data nyata, bukan placeholder kosong.
- Kreativitas dalam desain arsitektur agent, pemilihan tools, dan penyusunan prompt sangat dianjurkan.
- Seluruh kode harus ditulis dalam Python dan dapat dijalankan ulang oleh penguji setelah mengatur API key yang diperlukan.

2.2 Tools yang Direkomendasikan

Peserta tidak terbatas pada tools di bawah ini, namun seluruh tools berikut telah diajarkan selama kelas dan direkomendasikan untuk digunakan:

Kategori	Tools
Agent & Workflow Framework	LangChain, LangGraph (wajib sebagai framework utama)
LLM Provider	OpenAI (GPT-4o / GPT-4o-mini) — wajib untuk embedding & generasi
Vector Database	Qdrant Cloud — wajib sebagai penyimpanan vector embedding
Embedding Model	OpenAI text-embedding-3-small / text-embedding-ada-002
User Interface	Python built-in (input/print), rich atau colorama (opsional) — untuk terminal interface
Observability (Opsional)	LangFuse — untuk monitoring dan tracing eksekusi agent
Utilities	Python-dotenv, tiktoken, PyPDF2 / pdfplumber, python-docx

2.3 Catatan Teknis Penting

⚠ **Perhatian**

• Pembuatan agent dan workflow HARUS menggunakan LangChain dan LangGraph sebagai framework utama.

• Gunakan OpenAI service untuk embedding model dan Qdrant Cloud sebagai vector database.

• Pastikan tidak memasukkan data sensitif seperti API key ke dalam file yang di-upload ke GitHub.

- Gunakan file .env dan tambahkan ke .gitignore untuk menjaga keamanan credential.
- Setelah project di-submit, tidak diperbolehkan melakukan edit/update file. Jam last commit dianggap sebagai jam submit.
- Segala bentuk plagiarisme tidak akan ditoleransi dan mutlak diberikan nilai 0.

3. Task Description

Project dibagi menjadi empat komponen utama yang saling terintegrasi. Setiap komponen merepresentasikan satu lapisan dari arsitektur agentic RAG system yang lengkap.

> **Catatan Penting**
> Seluruh komponen bersifat wajib untuk diselesaikan. Penilaian mempertimbangkan kualitas implementasi dan integrasi antar komponen, bukan sekadar keberadaan fitur.
> Siswa yang hanya mengikuti contoh minimum tanpa eksplorasi mendapat nilai maksimum 75 per komponen.

Komponen 1 — Pembuatan Vector Database

> **Konteks**
> Sebelum chatbot dapat menjawab pertanyaan berbasis dokumen, sistem harus terlebih dahulu memproses dan menyimpan representasi semantik dari seluruh dokumen ke dalam vector database. Proses ini adalah fondasi dari seluruh sistem RAG yang akan dibangun.

Yang Harus Dibangun

1. Persiapan Data Dokumen: Siapkan minimal 3 dokumen dengan format yang beragam (PDF, TXT, atau DOCX) yang merepresentasikan konteks KnowledgeHub — misalnya: SOP onboarding karyawan, kebijakan cuti, atau dokumentasi teknis suatu fitur produk.
2. Document Loading & Parsing: Implementasikan mekanisme loading dokumen menggunakan LangChain Document Loaders yang sesuai untuk setiap format file.
3. Text Chunking: Terapkan strategi chunking yang tepat menggunakan RecursiveCharacterTextSplitter atau metode chunking lainnya. Pertimbangkan chunk_size dan chunk_overlap yang optimal untuk konteks dokumenmu.
4. Embedding Generation: Gunakan OpenAI Embedding Model untuk menghasilkan vector representation dari setiap chunk dokumen.
5. Qdrant Cloud Integration: Lakukan insert vector beserta metadata (source, page, chunk_id) ke Qdrant Cloud collection yang telah dikonfigurasi dengan akun masing-masing.
6. Validasi: Implementasikan langkah validasi sederhana untuk memastikan data berhasil tersimpan — misalnya dengan melakukan similarity search awal dan menampilkan hasilnya.

Output yang Diharapkan

- Script Python yang dapat dijalankan ulang untuk mengisi ulang vector database.
- Qdrant Cloud collection yang aktif dan terisi data dari minimal 3 dokumen.

- Bukti hasil validasi similarity search (screenshot atau output terminal).

Komponen 2 — Pembuatan RAG Tool

> **Konteks**
> RAG Tool adalah jembatan antara pertanyaan pengguna dan informasi yang tersimpan di vector database. Tool ini akan dipanggil oleh agent setiap kali pertanyaan membutuhkan konteks dari dokumen — bukan dari pengetahuan umum LLM semata.

Yang Harus Dibangun

1. Query Embedding: Implementasikan fungsi untuk mengubah query teks dari pengguna menjadi vector representation menggunakan model embedding yang sama dengan yang digunakan saat indexing.
2. Similarity Search: Lakukan pencarian ke Qdrant Cloud untuk menemukan top-k chunk dokumen yang paling relevan dengan query. Implementasikan filter metadata jika diperlukan.
3. Context Formatting: Format hasil retrieval menjadi konteks teks yang terstruktur dan siap digunakan oleh LLM untuk menghasilkan jawaban.
4. LangChain Tool Definition: Bungkus seluruh proses di atas menjadi sebuah LangChain Tool yang dapat dipanggil oleh agent, lengkap dengan nama tool, deskripsi yang jelas, dan schema input/output.
5. Source Attribution: Pastikan RAG tool menyertakan informasi sumber dokumen (nama file, nomor halaman) dalam outputnya agar jawaban dapat diverifikasi.

Output yang Diharapkan

- LangChain Tool object yang siap diintegrasikan ke dalam agent.
- Demonstrasi RAG tool berjalan secara standalone: input query → output context + source.

Komponen 3 — Pembuatan Multi-Agent

> **Konteks**
> Inti dari sistem ini adalah arsitektur multi-agent yang mampu mengkoordinasikan beberapa agen spesialis untuk menangani berbagai jenis pertanyaan. Supervisor agent bertanggung jawab untuk menilai pertanyaan dan mendelegasikan ke agent yang paling tepat.

Yang Harus Dibangun

1. Supervisor Agent: Implementasikan supervisor agent menggunakan LangGraph yang bertugas menganalisis pertanyaan pengguna dan menentukan routing ke sub-agent atau tool yang tepat.
2. RAG Agent: Bangun agent spesialis yang terintegrasi dengan RAG Tool (Komponen 2) untuk menjawab pertanyaan berbasis dokumen internal.
3. General Knowledge Agent: Bangun agent yang menangani pertanyaan umum yang tidak memerlukan konteks dokumen — menggunakan pengetahuan LLM secara langsung.
4. LangGraph State & Workflow: Definisikan state schema yang mencakup: messages, chat_history, current_agent, dan metadata relevan lainnya. Implementasikan graph flow yang menghubungkan supervisor dan sub-agents.
5. Chat History Integration: Pastikan seluruh agent dapat mengakses dan memanfaatkan riwayat percakapan. Minimal 3 percakapan ke belakang harus dipertahankan dalam konteks.
6. Transparency Output: Setiap eksekusi agent harus menghasilkan output yang menampilkan reasoning process — misalnya: agent mana yang aktif, tool apa yang dipanggil, dan hasil antara sebelum jawaban final.

Contoh Arsitektur Minimum

Agent / Node	Tanggung Jawab	Tools yang Digunakan
Supervisor Agent	Routing pertanyaan ke sub-agent yang tepat	—
RAG Agent	Menjawab pertanyaan berbasis dokumen internal	RAG Tool (Qdrant)
General Agent	Menjawab pertanyaan umum / percakapan	LLM langsung

Output yang Diharapkan

- LangGraph graph yang dapat dijalankan dengan minimal 2 sub-agent fungsional.
- Demonstrasi routing yang benar: pertanyaan berbasis dokumen → RAG Agent; pertanyaan umum → General Agent.
- Log transparansi yang menampilkan alur agent selama eksekusi.

Komponen 4 — Penyusunan Prompt

Konteks

Prompt adalah instruksi yang menentukan karakter, kapabilitas, dan batasan setiap agent. Prompt yang buruk akan menghasilkan routing yang salah, jawaban yang tidak relevan,

atau agent yang tidak konsisten. Komponen ini menguji kemampuan prompt engineering siswa.

Yang Harus Dibangun

1. System Prompt Supervisor: Tulis system prompt yang mendefinisikan peran supervisor secara jelas, kriteria routing yang spesifik, dan instruksi penanganan edge case (pertanyaan ambigu, pertanyaan di luar konteks, dll).
2. System Prompt RAG Agent: Tulis prompt yang menginstruksikan RAG agent untuk selalu menjawab berdasarkan konteks yang diberikan, menyebutkan sumber, dan jujur ketika informasi tidak ditemukan dalam dokumen.
3. System Prompt General Agent: Tulis prompt yang membatasi ruang lingkup agent ini dan memastikan konsistensi nada/gaya dengan agent lainnya.
4. Few-Shot Examples (Opsional, untuk poin tambahan): Sertakan 2–3 contoh pertanyaan-jawaban di dalam prompt untuk meningkatkan konsistensi output.
5. Prompt Versioning: Simpan seluruh prompt dalam file terpisah (misalnya prompts.py atau prompts.yaml) untuk kemudahan maintenance dan eksperimen.

Output yang Diharapkan

- File prompt yang terstruktur dan terdokumentasi dengan baik.
- Penjelasan singkat (dalam komentar kode atau README) tentang keputusan desain di balik setiap prompt.

Komponen 5 — User Interface & Experience (Terminal)

Konteks

Seluruh komponen yang telah dibangun harus diintegrasikan ke dalam antarmuka terminal yang clean, informatif, dan nyaman digunakan. Interface ini adalah wajah dari seluruh sistem — dan cerminan kemampuan siswa merancang alur aplikasi yang logis serta transparan bagi pengguna yang berinteraksi langsung melalui command line.

Yang Harus Dibangun

1. Loop Percakapan Interaktif: Implementasikan loop input-output di terminal yang memungkinkan pengguna mengetikkan pertanyaan secara berulang. Sertakan opsi keluar yang jelas (misalnya: ketik 'quit' atau 'q' untuk mengakhiri sesi).
2. Tampilan Proses Berpikir Agent: Setiap kali agent bekerja, tampilkan informasi proses secara real-time di terminal — termasuk: agent atau node mana yang sedang aktif, tool apa yang sedang dipanggil, dan ringkasan dokumen yang sedang diambil sebagai referensi.
3. Pemisah Visual yang Jelas: Gunakan separator, label, atau formatting teks (misalnya: garis pembatas, prefix [USER] / [AGENT] / [RAG], atau warna teks menggunakan library

seperti rich atau colorama) untuk membedakan antara input pengguna, proses agent, dan jawaban final.

4. Tampilan Sumber Dokumen: Setelah jawaban ditampilkan, sertakan informasi sumber dokumen yang digunakan (nama file, nomor halaman atau chunk) agar pengguna dapat memverifikasi informasi.
5. Penanganan Error yang Ramah: Tangani kondisi error (misalnya: pertanyaan tidak relevan, dokumen tidak ditemukan) dengan pesan yang informatif dan tidak membingungkan pengguna.
6. Chat History Display (Opsional, untuk poin tambahan): Sediakan perintah khusus (misalnya: ketik 'history') untuk menampilkan ringkasan riwayat percakapan dalam sesi yang sedang berjalan.

Contoh Tampilan Terminal yang Diharapkan

Contoh Output Terminal

```
==============================================================
 KnowledgeHub Assistant  |  Ketik 'q' untuk keluar
==============================================================
[USER] Apa kebijakan cuti tahunan karyawan?

  > Routing ke: RAG Agent
  > Memanggil tool: rag_search
  > Dokumen ditemukan: SOP_HR_Cuti.pdf (hal. 3), Kebijakan_Karyawan.txt

[AGENT] Berdasarkan dokumen internal, karyawan berhak atas 12 hari cuti
tahunan setelah masa kerja 1 tahun. Sumber: SOP_HR_Cuti.pdf, halaman 3.
==============================================================
```

Output yang Diharapkan

- Aplikasi terminal yang dapat dijalankan dengan perintah sederhana (misalnya: python main.py).
- Tampilan proses berpikir agent yang informatif dan mudah dibaca saat eksekusi.
- Demonstrasi minimal 5 sesi percakapan dengan berbagai jenis pertanyaan (berbasis dokumen dan umum).

4. Poin Penilaian

4.1 Bobot Komponen

Komponen Penilaian	Bobot
Presentasi Penjelasan	25%
Pembuatan Vector Database	10%
Pembuatan RAG Tool	15%
Pembuatan Multi-Agent	20%
Penyusunan Prompt	10%
User Interface & Experience (Terminal)	20%

4.2 Detail Komponen Penilaian

Siswa akan diberikan contoh flow dari tiap komponen penilaian sebagai basis minimum. Jika siswa hanya membuat aplikasi yang sesuai dengan contoh flow tersebut, maka nilai maksimum yang didapat adalah 75 untuk tiap komponen. Untuk mendapatkan nilai lebih tinggi, siswa harus menambahkan kompleksitas yang tetap sesuai dengan materi yang dipelajari.

Presentasi Penjelasan (25%)

Siswa dapat menjelaskan tujuan aplikasi, arsitektur sistem secara keseluruhan, dan fungsi aplikasi dan penjelasan setiap blok kode yang signifikan dengan cara yang jelas, terstruktur, dan mudah dipahami dan tepat waktu. Penjelasan harus mencerminkan pemahaman mendalam, bukan sekadar membaca kode serta dapat menjawab pertanyaan dan respon dari audiens.

Pembuatan Vector Database (10%)

Siswa dapat membangun pipeline pemrosesan dokumen — mulai dari loading, chunking, embedding, hingga penyimpanan ke Qdrant Cloud — dengan konfigurasi yang tepat dan dapat direproduksi.

Pembuatan RAG Tool (15%)

Siswa dapat membuat RAG tool yang fungsional dan dapat dipanggil oleh agent. Tool harus menghasilkan konteks yang relevan beserta informasi sumber yang dapat diverifikasi.

Pembuatan Multi-Agent (20%)

Siswa dapat membangun sistem multi-agent menggunakan LangGraph dengan routing yang akurat, state management yang baik, dan integrasi chat history yang fungsional. Transparansi proses berpikir agent menjadi nilai tambah yang signifikan.

Penyusunan Prompt (10%)

Siswa dapat merancang system prompt yang optimal untuk setiap agent — memastikan routing yang tepat, jawaban yang akurat berbasis dokumen, dan konsistensi perilaku agent.

User Interface & Experience — Terminal (20%)

Siswa dapat membuat interface terminal yang clean dan mudah dimengerti pengguna. Penilaian mencakup bagaimana siswa merancang alur aplikasi, kejelasan informasi yang ditampilkan saat agent sedang bekerja, serta kerapian output percakapan secara keseluruhan.

5. Rubrik Penilaian

Setiap komponen akan dinilai berdasarkan skala berikut:

Level	Deskripsi	Nilai
Excellent	Seluruh komponen berjalan sempurna, integrasi antar komponen solid, penggunaan LangChain & LangGraph sangat efektif, dan terdapat inovasi yang melampaui requirement minimum. Presentasi penjelasan disampaikan dengan percaya diri dan mendalam.	100
Very Good	Hampir semua komponen berjalan baik, integrasi jelas dan konsisten, prompt engineering dilakukan dengan baik. Presentasi penjelasan jelas dan terstruktur.	90
Good	Sebagian besar komponen fungsional dan terintegrasi. RAG dan agent bekerja dengan baik meski belum dioptimalkan. Presentasi penjelasan menyampaikan alur sistem dengan cukup baik.	80
Above Average	Komponen dasar berjalan dengan integrasi minimal. Agent dapat melakukan routing dasar. Presentasi ada namun kurang mendalam.	70
Average	Komponen hanya berjalan sebagian. Integrasi tidak stabil. Presentasi kurang terstruktur.	60
Poor	Banyak komponen wajib tidak berfungsi. Agent tidak dapat melakukan routing yang benar. Presentasi tidak jelas.	40
Very Poor	Tidak ada komponen yang dapat dijalankan. Tidak hadir atau tidak mengumpulkan deliverables.	20

6. Deliverables

Peserta wajib mengumpulkan seluruh item berikut pada tanggal yang telah ditentukan:

#	Item	Format	Keterangan
1	Source Code Python	GitHub Repository	Seluruh kode dapat dijalankan ulang. Sertakan README.md dengan instruksi setup dan cara menjalankan aplikasi terminal.
2	Bukti Eksekusi	Screenshot / Screen Recording	Setiap komponen harus didemonstrasikan berjalan dengan data nyata. Sertakan bukti similarity search, routing agent, chat history, dan tampilan terminal saat aplikasi berjalan.
3	Presentasi Penjelasan	Link YouTube / Google Drive (public access)	Maksimal 45 menit. Termasuk dalam proses penjelasan tanya jawab.
4	Google Form Submission	Google Form yang disediakan operasional	Cantumkan link GitHub dan video. Pastikan menerima email konfirmasi.

7. Waktu Pengerjaan & Pengumpulan

7.1 Durasi

Lama waktu pengerjaan Final Project adalah 10 hari kerja terhitung sejak tanggal distribusi dokumen ini.

7.2 Metode Pengumpulan

1. Upload seluruh file Python project (.py) dan file pendukung ke GitHub masing-masing. Pastikan API key tidak tersimpan dalam repository.
2. Isi Google Forms yang disediakan oleh operasional dengan mencantumkan link GitHub.
3. Pastikan menerima email konfirmasi pengumpulan. Cek folder spam jika email tidak ada di inbox.

> ⚠ **Penting**
> Setelah project di-submit, dilarang melakukan edit/update file project.
> Jika terdapat perubahan setelah submit, jam last update akan dianggap sebagai jam submit.
> Segala bentuk plagiarisme tidak akan ditoleransi dan mutlak diberikan nilai 0.

8. Panduan Kompleksitas Tambahan

Untuk meraih nilai di atas 75 per komponen, siswa dapat menambahkan kompleksitas berikut. Seluruh tambahan harus tetap relevan dengan materi yang telah dipelajari:

Komponen	Contoh Kompleksitas Tambahan
Vector Database	Implementasi multiple collection untuk dokumen dari domain berbeda; metadata filtering yang lebih kaya; re-indexing otomatis saat dokumen baru di-upload.
RAG Tool	Hybrid search (dense + sparse/BM25); re-ranking hasil retrieval menggunakan cross-encoder; penanganan dokumen multi-format secara dinamis.
Multi-Agent	Tambahan sub-agent spesialis (misalnya: SQL agent, calculator agent); implementasi human-in-the-loop checkpoint; agent memory jangka panjang.
Prompt	Chain-of-thought prompting untuk supervisor; few-shot examples berbasis domain spesifik; prompt yang menghasilkan jawaban terstruktur (JSON/Markdown).
Terminal UI	Implementasi warna dan formatting teks menggunakan library rich; mode streaming respons kata per kata di terminal; perintah interaktif tambahan (misalnya: 'history', 'clear', 'help'); progress bar saat proses retrieval berjalan.

9. Catatan & FAQ

Pertanyaan	Jawaban
Apakah boleh menggunakan framework selain LangChain/LangGraph?	Boleh sebagai tools pendukung, tetapi LangChain dan LangGraph wajib menjadi framework utama untuk pembuatan agent dan workflow.
Apakah data dokumen harus berupa data nyata?	Tidak harus, tetapi data dummy harus realistis dan representatif — bukan sekadar placeholder lorem ipsum.
Berapa jumlah sub-agent minimum?	Minimum 2 sub-agent: RAG Agent dan General Knowledge Agent. Supervisor tidak dihitung sebagai sub-agent.
Apakah boleh menggunakan model selain OpenAI?	OpenAI wajib digunakan untuk embedding. Untuk LLM generasi, boleh bereksperimen dengan model lain sebagai kompleksitas tambahan, tetapi harus ada fallback ke OpenAI.
Bagaimana jika Qdrant Cloud tidak dapat diakses?	Qdrant Cloud adalah requirement wajib. Hubungi pengajar segera jika ada kendala akses sebelum mendekati deadline.